

华中科技大学

算法设计与分析

题目： 算法设计与分析实验报告

专业班级 智科 2402

学 号 U202490035

姓 名 阮云轩

指导教师 丁俊文

报告日期 2026 年 1 月 1 日

目 录

1	引言	1
2	完成情况	2
3	P1228 地毯填补问题.....	4
3.1	题目分析	4
3.2	算法设计	4
3.3	性能分析	6
3.4	运行测试	7
4	P1854 [IOI 1999] 花店橱窗布置	8
4.1	题目分析	8
4.2	算法设计	9
4.3	性能分析	11
4.4	运行测试	11
5	P1443 马的遍历	12
5.1	题目分析	12
5.2	算法设计	12
5.3	性能分析	14
5.4	运行测试	15
6	P1250 种树.....	16
6.1	题目分析	16
6.2	算法设计	16
6.3	性能分析	19
6.4	运行测试	19
7	P1111 修复公路.....	21
7.1	题目分析	21
7.2	算法设计	21
7.3	性能分析	23
7.4	运行测试	24
8	总结	25

1 引言

本次算法设计与分析实践课程的实验报告，聚焦于将课堂所学的核心算法思想与数据结构应用到实际的编程问题求解中。通过在洛谷完成一系列题目，实现并测试了多种经典算法策略，包括分治法、贪心法、搜索算法（如 DFS、BFS 及回溯搜索）、DP 动态规划，并进一步结合进阶数据结构线段树等结构来处理区间查询与修改的高效需求。这些算法与数据结构的选择，并非随机，而是紧密围绕实际问题对时间复杂度与空间复杂度的严格要求。在不同题目中，我们需要针对输入规模（通常达到 10^5 10^6 量级）设计出从 $O(n^2)$ 优化到 $O(n \log n)$ 甚至 $O(n)$ 的解法，并在实作过程中反复验证正确性、边界条件与性能瓶颈。

透过本次实践，我不仅巩固了对各种算法范式的理解，更深刻体会到：

如何从暴力枚举逐步过渡到优雅的分治或贪心策略

如何利用状态设计与转移方程，将复杂问题转化为动态规划的可计算形式

如何运用线段树这类支持区间操作的数据结构，突破朴素方法的性能限制

2 完成情况

以下是我在洛谷的选题以及完成情况，题单共 50 题，已完成 18 题，见下图
(2-1)(2-2)(2-3)

本次报告会对以下题目作分析

P1228 地毯填补问题

P1854 [IOI 1999] 花店橱窗布置

P1443 马的遍历

P1250 种树

P1111 修复公路

✓	P2678	[NOIP 2015 提高组] 跳石头
—	P1242	新汉诺塔
✓	P1228	地毯填补问题
—	P1220	关路灯
—	P4170	[CQOI2007] 涂色
✓	P1854	[IOI 1999] 花店橱窗布置
—	P4999	烦人的数学作业
—	P1437	[HNOI2004] 敲砖块
—	P1434	[SHOI2002] 滑雪
—	P4017	最大食物链计数
✓	P2018	消息传递
✓	P1106	删数问题
✓	P1080	[NOIP 2012 提高组] 国王游戏
—	P2512	[HAOI2008] 糖果传递
—	P2127	序列排序
✓	P1658	购物
✓	P1090	[NOIP 2004 提高组] 合并果子

图 2-1 list

华中科技大学课程实验报告

✓	P1443	马的遍历
✓	P1135	奇怪的电梯
—	P2895	[USACO08FEB] Meteor Shower S
—	P1825	[USACO11OPEN] Corn Maze S
—	P1433	吃奶酪
—	P1784	数独
—	P1141	01迷宫
—	P1019	[NOIP 2000 提高组] 单词接龙 (疑似错题)
✓	P2349	金字塔
—	P2324	[SCOI2005] 骑士精神
—	P5691	[NOI2001] 方程的解数
—	P3067	[USACO12OPEN] Balanced Cow Subsets G
✓	P3809	【模板】后缀排序
—	P2852	[USACO06DEC] Milk Patterns G
—	P2870	[USACO07DEC] Best Cow Line G
—	P3374	【模板】树状数组 1
—	P3368	【模板】树状数组 2
✓	P3605	[USACO17JAN] Promotion Counting P
—	P1816	忠诚

图 2-2 list

✓	P3373	【模板】线段树 2
—	P1904	天际线
—	P1993	小 K 的农场
—	P3275	[SCOI2011] 糖果
✓	P1250	种树
✓	P1111	修复公路
✓	P1196	[NOI2002] 银河英雄传说
—	P2024	[NOI2001] 食物链
—	P1197	[JSOI2008] 星球大战
✓	P2756	飞行员配对方案问题
—	P1314	[NOIP 2011 提高组] 聪明的质监员
—	P1726	上白泽慧音
—	P3379	【模板】最近公共祖先 (LCA)
—	P3865	【模板】ST 表 & RMQ 问题

图 2-3 list

3 P1228 地毯填补问题

相传在一个古老的阿拉伯国家里，有一座宫殿。宫殿里有个四四方方的格子迷宫，国王选择驸马的方法非常特殊，也非常简单：公主就站在其中一个方格子上，只要谁能用地毯将除公主站立的地方外的所有地方盖上，美丽漂亮聪慧的公主就是他的人了。公主这一个方格不能用地毯盖住，毯子的形状有所规定，只能有四种选择（如图3-1）：

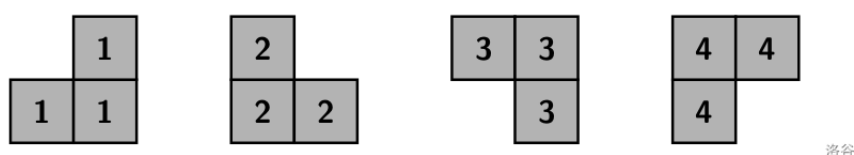


图 3-1 P1228

并且每一方格只能用一层地毯，迷宫的大小为 $2k \times 2k$ 的方形。当然，也不能让公主无限制的在那儿等，对吧？由于你使用的是计算机，所以实现时间为 1 秒。

3.1 题目分析

本题要求在一个边长为 $2k \times 2k$ 的方形棋盘（迷宫）上，除去公主所在的一个特殊格子（缺陷格）外，用四种特定形状的 L 型地毯（每块覆盖 3 个格子）完全覆盖剩余所有格子，且每格只能覆盖一次。分别对应四种地毯形状

问题本质是经典的「缺陷棋盘覆盖问题」（Defective Chessboard Tiling）的变形，使用 L-三连骨牌（tromino）覆盖 $2k \times 2k - 1$ 个格子。关键性质：总格子数 $4k - 1$ ，能被 3 整除，因此理论上可行。

从数学上用归纳证明可以推出存在合法解，用一般暴力法时间太大不够高效，我们需要换一种自相似的特性去处理这种问题

3.2 算法设计

采用分治 + 递归策略，将大问题不断拆分成四个规模为原问题一半的子问题。核心思想（与经典缺陷棋盘覆盖相同）：

将当前 $2m \times 2m$ 的棋盘划分为四个 $2m-1 \times 2m-1$ 的子棋盘。判断缺陷格（公

主位置)位于哪个象限。在四个象限的中心交界处放置一块 L 型地毯,使得另外三个没有缺陷格的象限各被覆盖一个格子,相当于给这三个子棋盘人为制造了一个缺陷格。这样,四个子棋盘都变成了「有一个缺陷格的较小棋盘覆盖问题」,可以递归处理。递归到规模为 1 (2×2 且已处理)时自然结束 (size==1 直接返回)。

在代码实现中:

坐标采用 0-based (方便计算象限),输入的 (x,y) 转为 (x-1, y-1) 每次递归记录当前放置的地毯: 拐角位置 (r,c) + 类型 c (1-4) 根据缺陷格所在象限,决定中心放置的地毯类型和位置递归时,对有真缺陷的子问题传原缺陷坐标;对另外三个子问题,传中心被覆盖的那个格子作为「假缺陷」

Listing 1 P1228.cpp

```
1      #include <bits/stdc++.h>
2      using namespace std;
3
4      vector<tuple<int, int, int>> bricks; // 存 (行, 列, c)
5      // t=top
6      // h=hole
7      // q=quadrant
8
9      void solve(int tr, int tc, int hr, int hc, int size) {
10         if (size == 1) return;
11
12         int half = size / 2;
13         int qr = (hr >= tr + half) ? 1 : 0;
14         int qc = (hc >= tc + half) ? 1 : 0;
15
16         int c, cr, cc; // 1-based
17         if (qr == 0 && qc == 0) { // 空格左上
18             c = 1; cr = tr + half + 1; cc = tc + half + 1;
19         } else if (qr == 0 && qc == 1) { // 空格右上
20             c = 2; cr = tr + half + 1; cc = tc + half;
21         } else if (qr == 1 && qc == 0) { // 空格左下
22             c = 3; cr = tr + half; cc = tc + half + 1;
23         } else { // 空格右下
24             c = 4; cr = tr + half; cc = tc + half;
25         }
26     }
```

```
27         bricks.emplace_back(cr, cc, c);
28
29         // 递归四个子区域
30         // 左上
31         int nhr = (qr == 0 && qc == 0) ? hr : tr + half - 1;
32         int nhc = (qr == 0 && qc == 0) ? hc : tc + half - 1;
33         solve(tr, tc, nhr, nhc, half);
34
35         // 右上
36         nhr = (qr == 0 && qc == 1) ? hr : tr + half - 1;
37         nhc = (qr == 0 && qc == 1) ? hc : tc + half;
38         solve(tr, tc + half, nhr, nhc, half);
39
40         // 左下
41         nhr = (qr == 1 && qc == 0) ? hr : tr + half;
42         nhc = (qr == 1 && qc == 0) ? hc : tc + half - 1;
43         solve(tr + half, tc, nhr, nhc, half);
44
45         // 右下
46         nhr = (qr == 1 && qc == 1) ? hr : tr + half;
47         nhc = (qr == 1 && qc == 1) ? hc : tc + half;
48         solve(tr + half, tc + half, nhr, nhc, half);
49     }
50
51     int main() {
52         int k, x, y;
53         cin >> k;
54         cin >> x >> y;
55         int size = 1 << k;
56         solve(0, 0, x - 1, y - 1, size); // 转为0-based
57
58         for (auto [r, c, tp] : bricks) {
59             cout << r << " " << c << " " << tp << endl;
60         }
61         return 0;
62     }
```

3.3 性能分析

时间复杂度:

每次递归处理当前规模的棋盘，并放置一块地毯 ($O(1)$)，然后递归四个子问题。递归树深度为 k ，每层总共放置的地毯数量约为 $(4^k - 1)/3 \approx 4^k / 3$ 。总共递归调用次数与放置的地毯数成正比，即 $O(4^k)$ 。但因为 $k \leq 10$ ， $4^{10} = 1048576 \approx 10^6$ ，时间完全可接受（远小于 1 秒限制）。

空间复杂度：

递归深度最多 $k = 10$ ，系统栈空间 $O(k) = O(1)$ 。储存所有地毯的 `vector` 大小为 $(4^k - 1)/3 \approx 349525$ ($k=10$ 时)，空间约几 MB，可接受。总结：时间与空间均与棋盘格子数成线性关系（实际上是 $\Theta(\text{格子数} / 3)$ ），属于高效解法。

3.4 运行测试

按照我们上方的思路与代码实现，我们设计如下的测试输入

3 ; 3 3 ;

```
/mnt/e/Document/CodingIDE/Clion/C++/C_Programming/cmake-build-wsldebug/bin/Luogu/Luogu_P1228_CarpetFillingProblem
3
3 3
5 5 1
2 2 4
1 1 4
1 4 3
4 1 2
4 4 1
2 7 3
1 5 4
1 8 3
3 6 3
4 8 1
7 2 2
5 1 4
6 3 2
8 1 2
8 4 1
7 7 1
6 6 1
5 8 3
8 5 2
8 8 1

进程已结束，退出代码为 0
```

图 3-2 P1228

4 P1854 [IOI 1999] 花店橱窗布置

某花店现有 F 束花，每一束花的品种都不一样。至少有同样数量的花瓶，被按顺序摆成一行。花瓶的位置是固定从左到右按 $1 \sim V$ 顺序编号， V 是花瓶的数目。

花束可以移动，并且每束花用 $1 \sim F$ 的整数标识。所有花束在放入花瓶时必须保持其标识数的顺序。例如，假设杜鹃花的标识数为 1，秋海棠的标识数为 2，康乃馨的标识数为 3，即杜鹃花必须放在秋海棠左边的花瓶中，秋海棠必须放在康乃馨左边的花瓶中。每个花瓶只能放一束花。

每个花瓶的形状和颜色也不相同，因此，当各个花瓶中放入不同的花束时，会产生不同的美学效果，并以美学值（一个整数 a_{ij}

）来表示，空置花瓶的美学值为 0。在上述的例子中，花瓶与花束的不同搭配所具有的美学值，可以用如下 (4-1) 的表格来表示：

	花瓶 1	花瓶 2	花瓶 3	花瓶 4	花瓶 5
杜鹃花	7	23	-5	-24	16
秋海棠	5	21	-4	10	23
康乃馨	-21	5	-4	-20	20

图 4-1 P1854

根据表格，杜鹃花放在花瓶 2 中，会显得非常好看，但若放在花瓶 4 中，则显得很难看。

为了取得最佳的美学效果，必须在保持花束顺序的前提下，使花的摆放取得最大的美学值。每个花束都必须被放入花瓶。如果具有最大美学值的摆放方式不止一种，则输出任何一种方案即可。

4.1 题目分析

本题要求将 F 束不同品种的花（必须按 $1 \sim F$ 的严格顺序）分配到 V 个花瓶中，每个花瓶最多放一束花，且必须全部使用这 F 束花，同时保持花的相对顺序不变（即花 i 必须放在花 j 的左边， $i < j$ ）。每个花瓶 j 放置花 i 会产生一个美学值 $a[i][j]$ （可正可负可零），空瓶美学值为 0。目标是求出最大总美学值，并输出

其中一种最优放置方案（花瓶编号序列）。约束条件：

$1 \leq F \leq V \leq 100$ 花的顺序必须严格保持（这意味着放置位置是严格递增的）
每个花瓶至多放一朵花（但可以有空瓶）

这是一个典型的带顺序约束的分配问题，可以建模为：

从 V 个位置中选出 F 个位置 $p_1 < p_2 < \dots < p_F$ 将花 1 F 依次放入 $p_1 p_F$ 最大化 $\sum a[i][p_i]$

由于 F 和 V 均 ≤ 100 ，直接枚举所有组合 $C(V,F)$ 会过大（ $C(100,50) \approx 1029$ ），因此我们需要使用动态规划优化题目。

4.2 算法设计

定义状态：

$dp[i][j]$ 表示前 i 朵花（1 i ）全部放置完毕，且最后一朵花（第 i 朵）放在第 j 个花瓶时，能达到的最大美学值总和。

状态转移：

$dp[i][j] = \max dp[i-1][k] + a[i][j]$ (其中 $1 \leq k < j$)

即：第 i 朵花放在 j 号瓶，前 $i-1$ 朵花的最后一朵放在 k 号瓶（ $k < j$ ），然后加上当前放置的价值 $a[i][j]$ 。边界：

$dp[0][j] = 0$ （没有花时，任何位置都是 0） $dp[i][j]$ 初始为极小值（表示不可达）

最终答案：

$\max(dp[F][j])$ 对所有 $j = F..V$ 取最大值（因为至少需要 F 个瓶子，且最后一朵花可以放在任意 $\geq F$ 的位置）

方案记录：

使用 $pre[i][j]$ 记录 $dp[i][j]$ 是从哪个 k 转移来的最后从 $dp[F][best_j]$ 开始回溯，依次得到放置位置

Listing 2 P1854.cpp

```
1      #include<bits/stdc++.h>
2      using namespace std;
3      #define ll long long
4      const int N=105;
5      int v[N][N];
6      int dp[N][N]; //j瓶放到第i朵
```

```
7      int n,m;
8      int a,b,c;
9      int pre[N][N];
10     void print(int k,int pl){
11         if(k==0) return;
12         print(k-1,pre[k][pl]);
13         printf("%d ",pl);
14     }
15     int main(){
16         scanf("%d%d",&n,&m);
17         for(int i=1;i<=n;i++){
18             dp[i][0]=-2e9;
19             for(int j=1;j<=m;j++) scanf("%d",&v[i][j]),dp[i][j]=-2e9;
20         }
21         for(int j=0;j<=m;j++) dp[0][j]=0;
22         for(int j=1;j<=m;j++){
23             for(int i=1;i<=n;i++){
24                 for(int k=i-1;k<j;k++){
25                     if(dp[i][j]<dp[i-1][k]+v[i][j]){
26                         dp[i][j]=dp[i-1][k]+v[i][j];
27                         pre[i][j]=k;
28                     }
29                 }
30             }
31         }
32         int ans=-2e9,pl;
33         for(int j=1;j<=m;j++){
34             if(ans<dp[n][j]){
35                 ans=dp[n][j];pl=j;
36             }
37         }
38         printf("%d\n",ans);
39         print(n,pl);
40         return 0;
41     }
42     /*
43     3 5
44     7 23 -5 -24 16
45     5 21 -4 10 23
46     -21 5 -4 -20 20
47     */
```

4.3 性能分析

时间复杂度: $O(F \cdot V^2)$

$F, V \leq 100$, 最坏情况约 $100 \times 10000 = 1,000,000$ 次操作, 常数小

空间复杂度: $O(F \cdot V)$

dp 和 pre 数组各约 $100 \times 100 = 10,000$ 个 int, 总共约 80KB

4.4 运行测试

按照我们上方的思路与代码实现, 我们设计如下的测试

输入:

3 5

7 23 -5 -24 16

5 21 -4 10 23

-21 5 -4 -20 20

输出:

53

2 4 5

```
/mnt/e/Document/CodingIDE/Clion/C++/C_Programming/cmake-build-wsldebug/bin/Luogu/Luogu_P1854
3 5
7 23 -5 -24 16
5 21 -4 10 23
-21 5 -4 -20 20
53
2 4 5
进程已结束,退出代码为 0
|
```

图 4-2 P1854

5 P1443 马的遍历

有一个 $n \times m$ 的棋盘，在某个点 (x, y) 上有一个马，要求你计算出马到达棋盘上任意一个点最少要走几步。

5.1 题目分析

本题要求计算在一个 $n \times m$ 的棋盘上，从起始位置 (x, y) 出发的「马」（国际象棋中的「马」），到达棋盘上每一个格子所需的最少步数。如果某个格子无法到达，则输出 -1。马的走法为「日」字形跳跃，即每次移动的位移为以下 8 种之一：

$(\pm 1, \pm 2)$ $(\pm 2, \pm 1)$

题目特点：

棋盘规模： $1 \leq n, m \leq 400$

起点坐标： $1 \leq x \leq n, 1 \leq y \leq m$

输出要求：完整输出 n 行 m 列的矩阵，每个位置表示从起点到该点的最短步数（不可达为 -1）

无障碍物，纯粹图论最短路问题

这是一个典型的无权图单源最短路问题。由于每一步的权重均为 1，且图的出度固定为 8，我们适合使用 BFS（广度优先搜索）求解。

5.2 算法设计

核心思路：将棋盘上的每个格子视为图的一个节点，相邻关系由马的 8 种走法决定，使用 BFS 从起点开始进行层序遍历，记录到达每个节点的最短距离。

具体实现步骤：

读入棋盘大小 n, m 及起点 (sx, sy) （注意题目中 x 是行， y 是列，均从 1 开始计数）

定义马的 8 个移动方向（ $dx[], dy[]$ 数组）

使用二维数组 $dist[][]$ 记录到达 (i, j) 的最少步数，初始全部置为 -1

使用 $vis[][]$ 标记是否已经访问过（避免重复入队）

使用队列进行 BFS：

起点入队， $dist[sx][sy] = 0$ ， $vis[sx][sy] = true$

每次从队首取出节点，枚举 8 个方向

若新位置合法（在棋盘内）且未访问，则：

标记已访问

距离 = 当前距离 + 1

入队

BFS 结束后，`dist[][]` 即为答案矩阵

按行输出，每个数值占固定宽度（题目接受任意合理场宽，常用

代码中采用了 `struct Node int x, y, step;` 来储存节点信息，也可以使用 `pair` 或三元组，但 `struct` 方式清晰易读。

Listing 3 P1443.cpp

```
1      #include <bits/stdc++.h>
2      using namespace std;
3
4      const int MAXN = 405;
5
6      int n, m, sx, sy;
7      int dist[MAXN][MAXN]; // 到 (i,j) 的最少步数，初始 -1
8      bool vis[MAXN][MAXN];
9
10     int dx[8] = { -1, -2, -2, -1, 1, 2, 2, 1 }; // 马的8个方向
11     int dy[8] = { -2, -1, 1, 2, 2, 1, -1, -2 };
12
13     struct Node {
14         int x, y, step;
15     };
16
17     void bfs() {
18         queue<Node> q;
19         q.push({sx, sy, 0});
20         vis[sx][sy] = true;
21         dist[sx][sy] = 0;
22
23         while (!q.empty()) {
24             Node cur = q.front(); q.pop();
25             int x = cur.x, y = cur.y, step = cur.step;
26
27             for (int i = 0; i < 8; i++) {
28                 int nx = x + dx[i];
```

```
29         int ny = y + dy[i];
30         if (nx >= 1 && nx <= n && ny >= 1 && ny <= m &&
            !vis[nx][ny]) {
31             vis[nx][ny] = true;
32             dist[nx][ny] = step + 1;
33             q.push({nx, ny, step + 1});
34         }
35     }
36 }
37 }
38
39 int main() {
40     scanf("%d%d%d%d", &n, &m, &sx, &sy);
41
42     memset(dist, -1, sizeof(dist));
43     memset(vis, 0, sizeof(vis));
44
45     bfs();
46
47     for (int i = 1; i <= n; i++) {
48         for (int j = 1; j <= m; j++) {
49             printf("%-5d", dist[i][j]); // 左对齐, 宽5格, 美观输出
50         }
51         printf("\n");
52     }
53
54     return 0;
55 }
```

5.3 性能分析

时间复杂度:

节点数: $n \times m \leq 400 \times 400 = 160,000$

每节点出边数: 固定 8 BFS 总时间: $O(\text{节点数} + \text{边数}) = O(n \cdot m + 8 \cdot n \cdot m)$
= $O(n \cdot m)$ 最坏情况 $\approx 160,000$ 次操作, 常数极小

空间复杂度:

队列最坏情况储存所有节点 $\approx 160,000$ 个 Node (每个约 12-16 字节) $\approx 2-3$ MB

总空间远低于常见限制（通常 256 MB）

结论：时间与空间均为线性级别，对于 $n, m \leq 400$ 完全高效。

5.4 运行测试

输入：

3 3 1 1

输出：

0 3 2

3 -1 1

2 1 4

```
/mnt/e/Document/CodingIDE/Clion/C++/C_Programming/cmake-build-wsldebug/bin/Luogu/Luogu_P1443
3 3 1 1
0 3 2
3 -1 1
2 1 4

进程已结束，退出代码为 0
```

图 5-1 P1443

6 P1250 种树

路边的地区被分割成块，并被编号成 $1, 2, \dots, n$ 。每个部分为一个单位尺寸大小并最多可种一棵树。

每个居民都想在门前种些树，并指定了三个号码 b, e, t 。这三个数表示该居民想在地区 b 和 e 之间（包括 b 和 e ）种至少 t 棵树。

居民们想种树的各自区域可以交叉。你的任务是求出能满足所有要求的最少的树的数量。

6.1 题目分析

本题要求在 1 到 n 的 n 个连续地块上种树，每块地最多种一棵树。有 h 个居民，每个居民提出一个区间 $[b_i, e_i]$ 并要求在该区间内至少种 t_i 棵树。所有要求必须同时满足，求最少需要种多少棵树。

题目特点：

每个地块至多种一棵树 $\rightarrow$ 种树位置互斥，而区间要求是「至少」而非「恰好」，即多个区间要求可以重迭，我们目标是最小化总种树数量（即最大化「空地」数量）

这是一个经典的区间覆盖 + 贪心问题。

核心矛盾在于：当多个区间重迭时，如何分配有限的种树位置，才能用最少的树满足所有「至少」要求。

暴力枚举每个区间的具体种树位置组合不可行（ $h \leq 5000, n \leq 3e4$ ，组合爆炸）。

因此，需要找到一种贪心策略，在保证满足所有约束的前提下尽量少种树。

6.2 算法设计

采用从右往左扫描 + 贪心 + 线段树维护已种树位置的策略。

具体思路：

将所有居民的要求按左端点 b_i 从大到小排序（即从右往左处理）

对每个要求 $[l, r]$ 至少种 cnt 棵树：

先查询当前 $[l, r]$ 区间内已经种了多少棵树（ sum ）

如果 $\text{sum} \geq \text{cnt}$ ，则该要求已经被之前的种树满足，无需再种

如果 $\text{sum} < \text{cnt}$ ，还需要再种 $(\text{cnt} - \text{sum})$ 棵

贪心选择：在 $[l, r]$ 内最靠右的、尚未种树的位置依次种上（因为右边位置对后续左边的要求影响较小）

为了快速实现「查询区间已种树数」和「找到区间内第 k 个空位」两个操作，使用带懒目标线段树：

维护每个位置的状态：0（未种）/ 1（已种）

支持区间赋值（种一片树）

支持区间求和（已种数量）

支持查询「全局第 k 个 0」（即第 k 个空位的位置）

代码实现细节：

线段树 $\text{sum}[x]$ 表示子树范围内已种树的数量

$\text{tag}[x]$ 用于懒标区间赋值（全部种或全部不种，但本题只用赋值为 1）

$\text{kth0}(x, k, l, r)$ ：在当前子树中找到第 k 个 0 的位置（递归判断左子树 0 的个数）

$\text{ser}(x, a, b, v)$ ：区间 $[a, b]$ 赋值为 v （本题 $v=1$ ）

Listing 4 P1250.cpp

```
1      #include<bits/stdc++.h>
2
3      #define N 30010
4
5      #define H 5010
6
7      using namespace std;
8
9      int n,h,ans;
10
11     struct SEG{
12
13         #define lc (x<<1)
14
15         #define rc (x<<1|1)
16
17         int sum[N<<2],tag[N<<2];
18
19         void pushup(int x){sum[x]=sum[lc]+sum[rc];}
20
21         void pushdown(int x,int l,int r){
22
23             if(!tag[x]) return;
24
25             int mid=(l+r)>>1;
26
27             sum[lc]=(mid-l+1)*tag[x],sum[rc]=(r-mid)*tag[x];
28
29             tag[lc]=tag[rc]=tag[x],tag[x]=0;
30
31         }
32
33         int kth0(int x,int k,int l,int r){
34
35             if(l==r) return l;
```

```
19         pushdown(x,l,r);
20         int mid=(l+r)>>1,lef0=mid-l+1-sum[lc];
21         if(lef0>=k) return kth0(lc,k,l,mid);
22         else return kth0(rc,k-lef0,mid+1,r);
23     }
24     void ser(int x,int a,int b,int v,int l,int r){
25         if(a<=l&&r<=b) return tag[x]=v,sum[x]=(r-l+1)*v,void();
26         pushdown(x,l,r);
27         int mid=(l+r)>>1;
28         if(a<=mid) ser(lc,a,b,v,l,mid);
29         if(b>mid) ser(rc,a,b,v,mid+1,r);
30         pushup(x);
31     }
32     int query(int x,int a,int b,int l,int r){
33         if(a>b) return 0;
34         if(a<=l&&r<=b) return sum[x];
35         pushdown(x,l,r);
36         int mid=(l+r)>>1,ans=0;
37         if(a<=mid) ans+=query(lc,a,b,l,mid);
38         if(b>mid) ans+=query(rc,a,b,mid+1,r);
39         return ans;
40     }
41 }tr;
42 struct Seg{int l,r,cnt;}a[H];
43 signed main(){
44     cin>>n>>h;
45     for(int i=1;i<=h;i++) cin>>a[i].l>>a[i].r>>a[i].cnt;
46     sort(a+1,a+1+h,[](Seg a,Seg b){return a.l>b.l;});
47     for(int i=1;i<=h;i++){
48         int sum=tr.query(1,a[i].l,a[i].r,1,n);
49         if(sum<a[i].cnt){
50             ans+=a[i].cnt-sum;
51             int pos=tr.kth0(1,a[i].cnt-sum+a[i].l-1,1,n);
52             tr.ser(1,a[i].l,pos,1,1,n);
53         }
54     }
55     cout<<ans<<"\n";
56     return 0;
57 }
58
59 //右边往左边开始排
```

```
60 //排重迭的
61 //贪心
```

6.3 性能分析

时间复杂度:

排序 h 个要求: $O(h \log h)$

对每个要求:

查询 $[l, r]$ 已种数: $O(\log n)$

若需补种 k 棵: 每次调用 `kth0` 找位置 $O(\log n)$, 然后单点/区间更新 $O(\log n)$

最坏情况每个要求都补种, 且每次补 1 棵 $\rightarrow$ 总更新次数 $\leq n$

总时间: $O((h + n) \log n)$

$n \leq 3e4$, $h \leq 5e3$, $\log n \approx 15$, 总操作量 $\approx 10^5 \sim 10^6$ 级别, 高效

空间复杂度:

线段树: $4 \times 30010 \times \text{sizeof(int)} \approx 480 \text{ KB}$

要求数组: $O(h) \approx 20 \text{ KB}$

总空间远低于限制

6.4 运行测试

输入

9

4

1 4 2

1 4 2

4 6 2

8 9 2

3 5 2

输出

5

```
/mnt/e/Document/CodingIDE/Clion/C++/C_Programming/cmake-build-wsldebug/bin/Luogu/Luogu_P1250
9
4
1 4 2
1 4 2
4 6 2
8 9 2
3 5 25

进程已结束，退出代码为 0
```

图 6-1 P1250

7 P1111 修复公路

给出 A 地区的村庄数 N ，和公路数 M ，公路是双向的。并告诉你每条公路的连着哪两个村庄，并告诉你什么时候能修完这条公路。问最早什么时候任意两个村庄能够通车，即最早什么时候任意两条村庄都存在至少一条修复完成的道路（可以由多条公路连成一条道路）。

7.1 题目分析

本题描述地震后所有公路损坏，给出 N 个村庄、 M 条双向公路，每条公路有修复完成时间 t 。问最早什么时候使得任意两个村庄都能通过已修复完成的公路（可经过多条）互相连通，即整个图连通的最早时刻。如果即使所有公路修好后仍不连通，输出 -1。

题目核心：

每条公路有不同的修复完成时间 t

时间 t 时，所有 $t' \leq t$ 的公路都已修好，可通行

要求整个图（ N 个点）在某个时刻变成单连通分量

输出这个时刻的最小值（即最后一条使图连通的公路的 t ）

所以我们知道这就是一个经典的最小生成树 + 边按权重排序的应用场景，但权重不是成本，而是时间，且我们关心的是「使连通分量数从 N 减到 1 的最后一条边的时间」。

7.2 算法设计

采用 Kruskal 算法的变形，按修复时间从小到大排序边，逐步加入已修复的公路，并使用并查集维护连通分量。

读入 N （村庄数）、 M （公路数）

读入 M 条公路： x, y, t （连接 x 和 y ，修复完成时间 t ）

将所有公路按 t 升序排序（从最早修好到最晚修好）

初始化并查集：每个村庄独立，连通分量数 = N

依次遍历排序后的公路：

若两端点 x 和 y 不在同一连通分量（ $\text{find}(x) \neq \text{find}(y)$ ）

合并它们 (union)

连通分量数减 1

每次合并后检查：若连通分量数 == 1，则当前公路的 t 就是答案（因为这条公路是最后一条使图连通的边）

若遍历完所有边后连通分量仍 > 1，输出 -1

Listing 5 P1111.cpp

```
1      #include <bits/stdc++.h>
2      using namespace std;
3      int father[1005];
4      struct road
5      {
6          int x;
7          int y;
8          int t;
9      }r[100005];
10     int cmp(road x,road y)
11     {
12         return x.t<y.t;
13     }
14     int find(int x)
15     {
16         if(father[x]==x) return x;
17         else return find(father[x]);
18     }
19     void union_set(int x,int y)
20     {
21         int fx=find(x);
22         int fy=find(y);
23         father[fy]=fx;
24     }
25     int main()
26     {
27         int n,m;
28         cin>>n>>m;
29         for(int i=1;i<=n;i++)
30         {
31             father[i]=i;
32         }
33         for(int i=1;i<=m;i++)
```



```
34         {
35             cin>>r[i].x>>r[i].y>>r[i].t;
36         }
37         sort(r+1,r+1+m,cmp);
38         for(int i=1;i<=m;i++)
39         {
40             if(find(r[i].x)!=find(r[i].y))
41             {
42                 union_set(r[i].x,r[i].y);
43                 n--;
44             }
45             if(n==1)
46             {
47                 cout<<r[i].t;
48                 return 0;
49             }
50         }
51         cout<<"-1";
52         return 0;
53     }
54
55     //应从最快的开始修
56     //修一次就检查一次是否全部连通
57     //通了就合并
```

7.3 性能分析

时间复杂度:

读入 + 排序 M 条边: $O(M \log M)$

并查集操作:

总共 M 次 find + 最多 $N-1$ 次 union

使用路径压缩 + 按秩合并 (或本题简单实现的路径压缩) 后, 近似 $O(M \alpha(N))$,

$\alpha(N) \approx 4$

$N \leq 1000$, $M \leq 10^5$, 总时间 $\approx O(M \log M)$ 主导, 远低于 1 秒

空间复杂度: 并查集 father[]: $O(N) \approx 4 \text{ KB}$

总空间极小

7.4 运行测试

4 4 1 2 6 1 3 4 1 4 5 4 2 3

```
/mnt/e/Document/CodingIDE/Clion/C++/C_Programming/cmake-build-wsldebug/bin/Luogu/Luogu_P1111
4 4
1 2 6
1 3 4
1 4 5
4 2 3
5
进程已结束，退出代码为 0
```

图 7-1 P1111

8 总结

通过《算法设计与分析》实验课上的实践，我完成了多类经典算法题目的编码、调试与优化过程，了解到不同的算法思想和数据结构

分治法

相关题目：洛谷 P1228 地毯填补（缺陷棋盘 L-三连覆盖）

利用问题的自相似性，将大规模问题分解为四个子问题，并在每次分解时「人为制造缺陷」来保持递归不变性；熟练掌握二维坐目标象限判断与中心点放置策略。

动态规划

相关题目：P1854 [IOI1999] 花店橱窗布置

处理带严格顺序约束的分配问题时，如何定义「前 i 个物品放到第 j 个位置」的状态；掌握二维 DP + 前驱记录回溯路径的标准写法；理解负数价值时初始化为极小值的必要性。

图论搜索（BFS）

相关题目：P1443 马的遍历

将棋盘模型化为图，使用 BFS 求单源最短路；熟练处理八方向移动的坐标偏移；掌握二维 vis 数组与 dist 数组的配合使用，以及输出格式的对齐技巧。

贪心 + 线段树优化

相关题目：P1250 种树

从右往左扫描的贪心策略在「区间至少覆盖」问题中的正确性；线段树支持「区间赋值」「区间求和」「查询第 k 个 0」三种操作的综合应用；理解如何用数据结构弥补纯贪心难以精确定位的缺陷。

并查集 + Kruskal 变形

相关题目：P11111 修复公路

将「最小生成树」思想应用到「最早连通时间」问题；按时间排序后逐步加入边，并实时维护连通分量数的技巧；理解「最后一条使连通分量数从 N 变为 1 的边的权重」即为答案。

通过这学期的算法实践，我深刻感受到算法学习的核心关键，而在于理解每种范式的本质以及面对新问题时的建模能力。从最开始写暴力解法 TLE，到后来逐步学会分析数据范围、选择合适算法、优化时间复杂度，这个过程让我对

「如何思考算法题」有了更系统的认识。